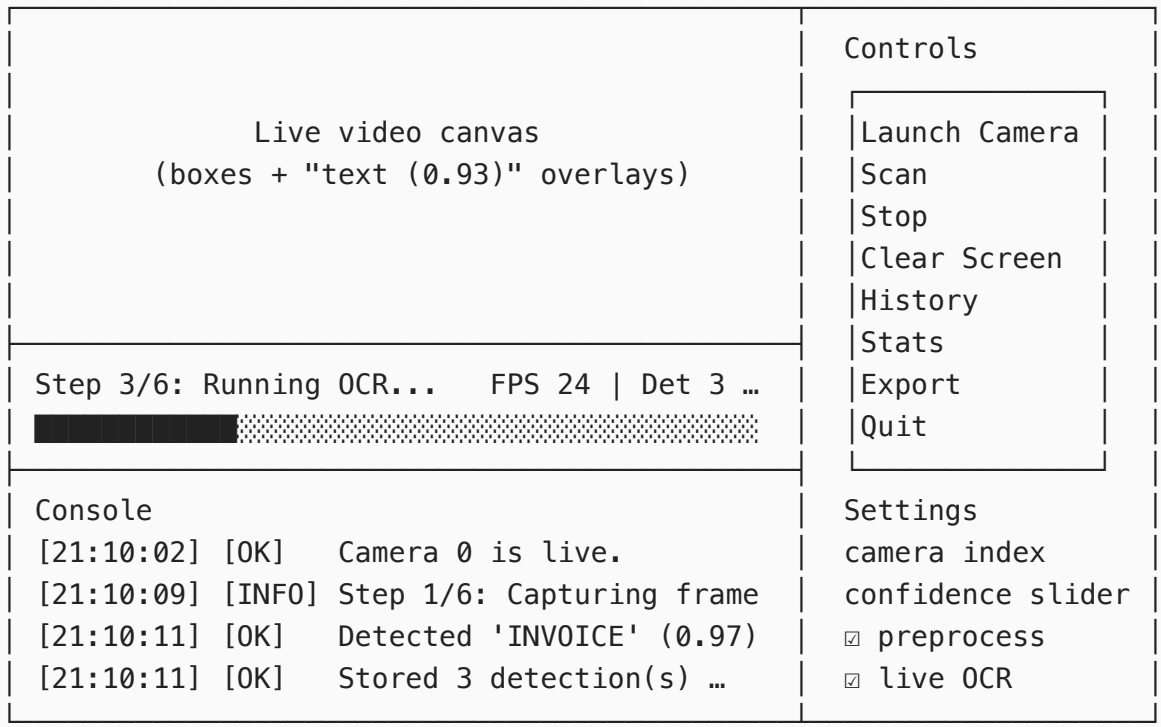
OCR Data Scanning Tool

A desktop application that turns a webcam into a real-time text scanner. It shows the live camera feed, runs **EasyOCR** on the frames in a background thread, overlays bounding boxes and recognised text, and - on demand - scans the current frame, saves an annotated image and
stores every detection in **MySQL** (with an automatic **SQLite offline fallback**). Past scans can be browsed, searched, exported (CSV / JSON / TXT), deleted and charted.



Contents

1. Features
2. Requirements
3. Setup, step by step
4. Running the app
5. Running the tests
6. Configuration
7. Troubleshooting
8. Screenshots
9. Architecture
10. Design decisions

11. Project layout

Features

- **Live preview** - webcam feed on a Tk canvas (aspect ratio preserved, "Camera Off" placeholder).
  OCR runs on every *N*th frame (default 10) and draws green boxes with `text (confidence)` labels.
- **Scan** - grabs the current frame, optionally preprocesses it (grayscale → denoise → CLAHE),
  runs full OCR, saves an annotated PNG to `captures/`, writes one row per detection to the database, logs every string to the console, beeps, and walks a progress bar through `Capturing frame → Preprocessing → Running OCR → Saving image → Writing to DB → Done`.
- **Never blocks the UI** - camera capture, OCR inference and camera opening each run on their own
  thread; the Tk thread only ever receives callables through a queue.
- **MySQL first, SQLite when MySQL is down** - the database and tables are created automatically on
  first run. If MySQL is unreachable the app keeps working and says so in the console.
- **History window** - Treeview of scans, substring search, multi-select delete (with confirmation), open the capture image, export the visible rows as CSV / JSON / TXT.
- **Stats window** - scans per day (last 30 days) and a confidence histogram (matplotlib embedded
  in Tk) plus headline numbers.
- **Confidence slider** (default 0.40), **preprocessing toggle**, **live-OCR toggle**, **camera index** selector, keyboard shortcuts **L** (launch) **S** (scan) **Esc** (stop) **Q** (quit).
- **Plain-language errors** for camera, OCR-model, database, file and audio problems; full tracebacks go to `logs/app.log`, never to the screen.

Requirements

	Minimum
OS	Windows 10/11, macOS 12+, Ubuntu 22.04+ (anything with a Tk-enabled Python)
Python	**3.10, 3.11 or 3.12** with `tkinter` available
Camera	Any built-in or USB webcam (UVC). Default index 0, selectable in the UI

	Minimum
CPU / GPU	CPU-only works (about 0.5-2 s per scan at 720p on a laptop). CUDA is used automatically when PyTorch reports it
RAM	4 GB free (PyTorch + EasyOCR models ≈ 1.5 GB)
Disk	~3 GB for the Python packages, ~100 MB for the English EasyOCR models (downloaded on first run)
Database	MySQL 8.x (5.7+ works). Optional - the app falls back to SQLite
Internet	Only once, to download the EasyOCR models

Setup, step by step

1. Install Python

- **Windows** - download 3.12 from https://www.python.org/downloads/. In the installer tick *Add python.exe to PATH* and keep *tcl/tk and IDLE* selected (that is `tkinter`).
- **macOS** - `brew install python@3.12 python-tk@3.12`, or use the python.org installer (which
  bundles Tk). See *Troubleshooting* if `pip` crashes on Homebrew Python.
- **Ubuntu / Debian** - `sudo apt install python3 python3-venv python3-tk libgl1`.

Check: `python3 -c "import tkinter; print(tkinter.TkVersion)"` prints a version number.

2. VS Code (recommended editor)

1. Install VS Code from https://code.visualstudio.com/.
2. Install the extensions **Python** (`ms-python.python`) and **Jupyter** (`ms-toolsai.jupyter`).
3. Open the project folder, then *Ctrl/Cmd+Shift+P → "Python: Select Interpreter"* and pick the virtual environment created in step 4 (`.venv`). The Jupyter extension will use the same interpreter for `notebooks/demo.ipynb`.
4. Optional: *Run → Add Configuration → Python File* and set `"program": "${workspaceFolder}/main.py"`.

3. MySQL Community Server + Workbench

1. Download **MySQL Installer** (Windows) or the DMG/DEB packages from https://dev.mysql.com/downloads/. Choose *MySQL Server 8.x* and *MySQL Workbench*.
   - macOS alternative: `brew install mysql && brew services start mysql`
   - Docker alternative (any OS):
     `docker run -d --name ocr-mysql -e MYSQL_ROOT_PASSWORD=root -p 3306:3306`

`mysql:8.4`

2. During installation set a **root password** and keep the default port **3306**.
3. Open Workbench, connect as `root`, and create a dedicated user (the application creates the database and tables itself):

```
CREATE USER 'ocr_user'@'localhost' IDENTIFIED BY 'change_me';
GRANT ALL PRIVILEGES ON ocr_scanner.* TO 'ocr_user'@'localhost';
FLUSH PRIVILEGES;
```

   Use `'ocr_user'@'%'` instead of `@'localhost'` if MySQL runs in Docker or on another machine.
4. Optional: run `database/schema.sql` in Workbench to create the schema manually.

4. Get the code and install the dependencies

```
git clone <this repository> ocr_scanner
cd ocr_scanner
python3 -m venv .venv
# Windows:  .venv\Scripts\activate
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt          # runtime + tests
pip install -r requirements-dev.txt      # optional: ruff + notebook tooling
```

`pip check` should report *No broken requirements found*.

5. Configure

```
cp .env.example .env
```

Edit `.env` and set at least `DB_USER` and `DB_PASSWORD`. Everything else has sensible defaults (see Configuration). Without a `.env` the app still starts, in offline mode.

Running the app

```
source .venv/bin/activate        # Windows: .venv\Scripts\activate
python main.py                   # or: python main.py --camera 1 --log-level
DEBUG
```

1. The console reports the database connection (`[OK] Connected to MySQL …` or a `[WARN]` offline

banner) and `[INFO] Loading EasyOCR models…`. The first ever run downloads the models (~100 MB); afterwards loading takes a few seconds. `[OK] OCR engine ready (CPU)` follows.

2. Press **Launch Camera** (or **L**). Boxes appear on the video as text is recognised.
3. Point the camera at printed text and press **Scan** (or **S**). Watch the progress bar and the console; the annotated frame lands in `captures/`, the rows in MySQL.
4. **History** browses/search/deletes/exports, **Stats** charts, **Export** saves everything as CSV/JSON/TXT (format follows the file extension you choose), **Clear Screen** empties the console, **Stop** (or **Esc**) releases the camera, **Quit** (or **Q**, or closing the window) shuts everything down cleanly.

Everything the app prints is also written to `logs/app.log` (rotating, 5 × 2 MB).

Running the tests

```
pytest -q                      # whole suite (119 tests)
pytest -q -m "not slow"        # skip the two tests that run the real EasyOCR
model
pytest -q -m "not gui"         # skip tests that need a Tk display (CI
without a screen)
ruff check . && ruff format --check .
python -m compileall -q main.py config core database gui utils tests
```

- **MySQL tests** (`-m mysql`) run automatically when a server is reachable with the credentials from `.env` (or `OCR_TEST_MYSQL_HOST/PORT/USER/PASSWORD`). They use a separate database `ocr_scanner_test`, which is dropped and recreated each run. Unreachable server ⇒ skipped.
- **Slow tests** (`-m slow`) render "HELLO WORLD" into an image with Pillow and assert EasyOCR reads it back - this is the end-to-end proof that OCR works on your machine.
- `tests/test_app_smoke.py` boots the full GUI headlessly with a synthetic camera and fake engine, performs a complete scan cycle, checks the database row, the saved image, the console lines and that no threads are left after *Quit*.

Configuration

All keys live in `.env` (real environment variables override the file).

Key	Default	Meaning
`DB_HOST` / `DB_PORT`	`localhost` / `3306`	MySQL server
`DB_USER` / `DB_PASSWORD`	`root` / *(empty)*	MySQL credentials - never hard-coded
`DB_NAME`	`ocr_scanner`	Database (created if missing; letters/digits/underscore only)
`CAMERA_INDEX`	`0`	Default camera; also changeable in the UI and via `--camera`
`CAMERA_WIDTH` / `CAMERA_HEIGHT`	`1280` / `720`	Requested capture size (the driver may pick the nearest)
`OCR_LANGUAGES`	`en`	Comma-separated EasyOCR codes, e.g. `en,hi`
`OCR_GPU`	`auto`	`auto` (CUDA if available) / `true` / `false`
`OCR_EVERY_N_FRAMES`	`10`	Run live-preview OCR on every Nth frame
`CONFIDENCE_THRESHOLD`	`0.4`	Initial slider value; lower detections are skipped
`PREPROCESS_ENABLED`	`true`	Initial state of the *Preprocess before scan* toggle
`OCR_MODEL_DIR`	*(unset → `~/.EasyOCR`)*	Where EasyOCR keeps its models
`AUDIO_ENABLED`	`true`	Beep on scan success/failure
`LOG_LEVEL`	`INFO`	`DEBUG` / `INFO` / `WARNING` / `ERROR`

Invalid values (e.g. `CAMERA_INDEX=-3`, `CONFIDENCE_THRESHOLD=7`) fall back to the default instead of
crashing.

Troubleshooting

Symptom	Cause / fix
`[ERROR] Camera 0 was not found or is already in use`	Wrong index (try *Camera index* = 1 or `--camera 1`), another app (Zoom/Teams) holds the camera, or the OS blocked access. **macOS**: System Settings → Privacy & Security → Camera → allow your terminal/VS

Symptom	Cause / fix
	Code. **Windows**: Settings → Privacy → Camera → allow desktop apps.
`[WARN] Camera 0 stopped delivering frames; trying to reconnect…`	Cable unplugged or driver hiccup. The app retries 3× (1 s apart); if it gives up you get an `[ERROR]` and can press *Launch* again.
`[WARN] Frame is blank – is the lens covered…`	The frame is almost black or uniform (lens cap, no light, camera still warming up).
`[WARN] No text detected`	Nothing above the confidence threshold. Move closer, improve lighting, lower the slider, or toggle *Preprocess*.
`[ERROR] EasyOCR could not be initialised… downloads about 100 MB of models`	First run needs internet (proxy? firewall?). Retry, or download `craft_mlt_25k.pth` + `english_g2.pth` from the EasyOCR model page into `~/.EasyOCR/model` (or the folder in `OCR_MODEL_DIR`).
`MySQL server is not reachable at localhost:3306`	Start the MySQL service (Windows *Services*, `brew services start mysql`, `docker start ocr-mysql`) or fix `DB_HOST` / `DB_PORT`. The app meanwhile runs in offline mode.
`MySQL rejected the credentials for user …`	Fix `DB_USER` / `DB_PASSWORD` in `.env`.
`User … lacks permission on database …`	Run the `GRANT ALL PRIVILEGES ON ocr_scanner.* …` statement from the setup section.
`[WARN] MySQL unavailable … Running in OFFLINE mode`	Expected when MySQL is down. Scans go to `data/offline_scans.sqlite3`; export them as CSV/JSON later if you want them in MySQL.
`ModuleNotFoundError: No module named '_tkinter'`	Python was built without Tk: `brew install python-tk@3.12` / `sudo apt install python3-tk` / re-run the Windows installer with *tcl/tk* ticked.
`pip` crashes with `ValueError: invalid literal for int() … mac_ver` or `Symbol not found:`	The Homebrew Python bottle links against a newer system `libexpat` than your macOS has. Use a python.org build or `uv venv --`

Symptom	Cause / fix
`_XML_SetAllocTrackerActivationThreshold` (macOS)	`python 3.12` instead (this is how the project was developed).
`[WARN] Audio feedback unavailable (no audio device)`	Harmless; pygame's mixer could not open a device. Set `AUDIO_ENABLED=false` to silence the warning.
Live preview is choppy on a slow laptop	Raise `OCR_EVERY_N_FRAMES`, untick *Live OCR overlay*, or lower `CAMERA_WIDTH` / `CAMERA_HEIGHT`. Scanning is unaffected.
`UserWarning: 'pin_memory' … not supported on MPS` / `quantize_per_tensor … deprecated`	Emitted by PyTorch inside EasyOCR on Apple Silicon; harmless.
Both `opencv-python` and `opencv-python-headless` are installed	EasyOCR depends on the headless build; both provide `cv2` and coexist fine.
macOS prints `objc[…]: Class SDLApplication is implemented in both … cv2/.dylibs/libSDL2 … and … pygame …` at start-up	OpenCV 5 and pygame each bundle SDL2; the runtime picks one. Cosmetic - the app is unaffected.
The window is too small / widgets overlap	Minimum size is 1100×700; the layout is grid-weighted, so just enlarge the window.

Screenshots

No screenshots are bundled: the acceptance run happened in an automated session without screen-recording permission, so `screencapture` produced empty files. The window layout is the
ASCII sketch at the top of this file. To add real ones, run the app and drop PNGs into `docs/screenshots/` with these names, then link them here:

File	What to show
`01_idle.png`	Main window before launching (placeholder canvas, console banner)
`02_live.png`	Live preview with green boxes and `text (confidence)` labels
`03_after_scan.png`	Progress bar at *Done* and the `[OK] Detected …` console lines
`04_history.png`	History window with a search applied
`05_stats.png`	Stats window (scans per day + confidence histogram)

One artefact from the acceptance run *is* included: the annotated frame the scan pipeline saved to `captures/` (synthetic 1280×720 input, real EasyOCR on CPU):

“docs/screenshots/sample_annotated_capture.png” could not be found.

Architecture

```
main.py ──► gui/app.py (OCRScannerApp, Tk main thread)
               │  owns: VideoCanvas · ControlPanel · ProgressPanel · ConsolePanel
               │        HistoryWindow · StatsWindow
               │
               ├─ core/camera.py      CameraManager  ── thread "camera-capture" ──► bounded frame queue
               ├─ core/ocr_worker.py  OCRWorker      ── thread "ocr-worker" ──► live detect() / scan task
               │      └─ core/ocr_engine.py  OCREngine (lazy easyocr.Reader)
               │      └─ core/scan_pipeline.py run_scan(): preprocess → OCR → PNG → DB
               ├─ core/detector.py    preprocess_frame · filter_by_confidence · draw_detections
               ├─ core/audio.py       AudioFeedback (pygame, silent fallback)
               └─ database/db_manager.py DatabaseManager ──► MySQLScanRepository | SQLiteScanRepository
```

Threading contract. Background threads never touch Tk. They call `OCRScannerApp._post(fn)`, which puts `fn` on a `queue.Queue`; `root.after(30 ms)` drains it on the main thread. The video loop (`root.after(33 ms)`) reads the newest camera frame, submits every Nth frame to the OCR worker (dropped if the worker is busy), draws the last live results and paints the canvas. A scan is a single task queued on the same worker, so live and scan inference never run concurrently. `quit_app()` cancels timers, joins the camera-opener and camera threads, stops the worker, closes the session row, closes the database pool, quits the mixer, calls `cv2.destroyAllWindows()` and finally destroys the window - also on the window-manager close button.

Design decisions

Choices made where the brief was silent or left room:

1. **Offline fallback is SQLite, not JSONL.** It implements the exact same `ScanRepository` interface, so History/Stats/Export keep working offline. File: `data/offline_scans.sqlite3`.
2. **Search uses `LIKE '%term%'`** (case-insensitive, wildcards escaped) rather than MySQL `FULLTEXT`, because full-text search ignores short tokens and partial words - unhelpful for OCR fragments like "A-17". The `FULLTEXT` index from the brief's schema is still created.
3. **One OCR worker thread serialises live preview and scans.** EasyOCR is not thread-safe and
   CPU-bound; running two inferences at once would only slow both down. Live frames are dropped
   while the worker is busy (never queued), scans are always queued. Live OCR is paused during a
   scan.
4. **Live preview feeds the raw BGR frame** to EasyOCR; the preprocessing toggle applies to *Scan* only. Preprocessing (Non-local-means denoise + CLAHE) costs ~100 ms at 720p, which is
   fine once per scan but would halve the live frame rate.
5. **Preprocessing defaults:** grayscale → `fastNlMeansDenoising(h=10)` → CLAHE (clip 2.0, 8×8).
   Adaptive thresholding and resizing exist (`PreprocessOptions`) but are off by default because
   binarisation usually *hurts* EasyOCR on natural camera images.
6. **Zero detections store nothing** - no PNG, no rows - and produce a `[WARN] No text detected`
   plus a low beep. A blank/black frame is detected before OCR (`mean < 8` or `std < 3`) and reported as such.
7. **Camera opening runs on a helper thread on Windows/Linux** (`STARTING` state) because `cv2.VideoCapture` can block for 1-3 s. **On macOS it runs on the Tk main thread**: OpenCV's
   AVFoundation backend can only display the system camera-permission prompt from the main
   thread and otherwise fails with "not authorized to capture video" (found during the acceptance
   run). The one-second stall on *Launch* is the price of a working first-run experience.
   Reconnect policy: 5 consecutive failed reads → up to 3 reopen attempts, 1 s apart → `[ERROR]` and back to *Idle*.
8. **Frame hand-off is a 2-slot queue where the newest frame wins**; the UI never displays stale
   frames and memory cannot grow.

9. **State machine** `IDLE → STARTING → RUNNING ↔ SCANNING`. *Scan* is enabled only in RUNNING,
   Stop in STARTING/RUNNING, *Launch* only in IDLE; History/Stats/Export/Clear/Quit always work.
   The camera-index spinbox is locked while the camera runs.
10. **Status line** shows FPS (smoothed, from the capture thread), the number of detections in the latest OCR result (live or scan), the highest confidence of that result, camera index and database backend.
11. **Sessions**: a `scan_sessions` row is created on *Launch* and closed on *Stop*/*Quit* with the number of detections stored during it. Deleting a session nulls `scans.session_id` (FK `ON DELETE SET NULL`, also in SQLite).
12. **Timestamps are naive local time** (`DATETIME` has no timezone); the History window shows them
    as `YYYY-MM-DD HH:MM:SS`.
13. **Export**: the main-window *Export* button opens a save dialog and infers the format from the extension (unknown → CSV). The History window exports the *currently visible* rows straight into `exports/scans_<timestamp>.<fmt>` without a dialog. CSV has a header row and JSON-encoded
    bbox; JSON is a list of objects with ISO timestamps; TXT is `[time] (conf) text` per line.
14. **Delete asks for confirmation** (messagebox) - the only modal dialog besides the export picker and the fatal-startup error.
15. **Database bootstrap**: `CREATE DATABASE IF NOT EXISTS` (name validated as an identifier, because identifiers cannot be bound as parameters), then `schema.sql` is executed statement by
    statement. If the user may not create databases but the database exists, that is accepted. A 3-connection pool serves the main and worker threads.
16. **`OCR_GPU=auto` only means CUDA.** EasyOCR does not use Apple MPS or DirectML; those machines run on CPU. The device is shown in the console (`OCR engine ready (CPU)`).
17. **Everything drawn programmatically** - the window icon is rendered with Pillow at start-up, the theme is a dark `clam`-based ttk style; no binary assets to ship.
18. **Keyboard shortcuts are ignored while a text field has focus** (search box, camera spinbox)
    so typing "s" does not trigger a scan.
19. **Logging**: `logs/app.log`, rotating 2 MB × 5, plus WARNING+ mirrored to stderr. Every caught
    exception is logged with `logger.exception` (traceback) and surfaced as one plain sentence.
20. **Pinned dependencies** are the exact versions used for development on 2026-09-20 (EasyOCR 1.7.2, OpenCV 5.0, PyTorch 2.14, Pillow 12.3, NumPy 2.5, mysql-connector

26.7).
`torch` from PyPI is the default CPU/CUDA wheel; nothing else is required for CPU machines.

21. **Tests never need hardware or internet except the two `slow` tests** (real EasyOCR) and the
`mysql` tests (auto-skip). Camera tests inject a fake `cv2.VideoCapture`; the app smoke test
injects a fake camera and engine and a MySQL connector that is "down" to exercise the fallback.
22. **Development environment note**: on this macOS 26 machine the Homebrew Python 3.12 bottle
has a broken `pyexpat`, so the project venv was created with `uv venv --python 3.12` (CPython 3.12.13). MySQL 8.4 ran in Docker for the integration tests.

Project layout

```
ocr_scanner/
├── main.py                    # entry point (argparse, logging, Tk root,
error dialog)
├── requirements.txt           # pinned runtime + test dependencies
├── requirements-dev.txt       # ruff, nbformat, nbconvert, ipykernel
├── pyproject.toml             # ruff + pytest configuration
├── .env.example               # copy to .env and fill in MySQL credentials
├── README.md · CHANGELOG.md
├── config/settings.py         # Settings dataclass loaded from .env with
validation
├── core/
│   ├── camera.py              # CameraManager (thread, bounded queue,
reconnect, FPS)
│   ├── ocr_engine.py          # OCREngine + OCRResult (lazy EasyOCR, GPU
auto-detect)
│   ├── ocr_worker.py          # OCRWorker thread (live jobs dropped when
busy, tasks queued)
│   ├── detector.py            # preprocess_frame, filter_by_confidence,
scale_results, draw_detections
│   ├── scan_pipeline.py       # run_scan(): the 6-step scan
│   └── audio.py               # AudioFeedback (pygame beeps, silent
fallback)
├── database/
│   ├── schema.sql             # MySQL DDL (also executed automatically)
│   ├── models.py              # ScanRecord, SessionRecord, ScanRepository
protocol, DatabaseError
│   ├── mysql_repository.py    # pooled MySQL implementation, auto-create
DB/tables, error mapping
```

```
│   ├── sqlite_repository.py   # offline implementation (same interface)
│   └── db_manager.py          # DatabaseManager: MySQL → SQLite fallback
├── gui/
│   ├── app.py                 # OCRScannerApp: wiring, state machine,
threads → Tk queue
│   ├── control_panel.py       # buttons + settings, AppState
│   ├── console_panel.py       # Listbox + Scrollbar log with levels
│   ├── progress_panel.py      # Progressbar + status label
│   ├── video_canvas.py        # aspect-preserving frame display
│   ├── history_view.py        # Treeview, search, delete, export, open
image
│   ├── stats_view.py          # matplotlib charts (FigureCanvasTkAgg)
│   └── theme.py               # palette, fonts, ttk styles, generated icon
├── utils/
│   ├── logger.py              # rotating file logging
│   └── helpers.py             # timestamps, blank-frame check, fit_within,
export_records
├── notebooks/demo.ipynb       # EasyOCR walkthrough on a synthetic image
(executed)
├── tests/                     # 119 tests (pytest, pytest-mock)
├── docs/                      # spec, implementation plan, screenshots
├── captures/ exports/ logs/ data/   # created at runtime (git-ignored)
└── .venv/                     # your virtual environment (git-ignored)
```

The notebook `notebooks/demo.ipynb` walks through EasyOCR on a generated image (raw `readtext`
output, OpenCV boxes, the project's `OCREngine`, confidence thresholds, and saving through the
repository layer). Open it in VS Code with the `.venv` kernel or run
`jupyter nbconvert --to notebook --execute notebooks/demo.ipynb`.